QUESTION-21:

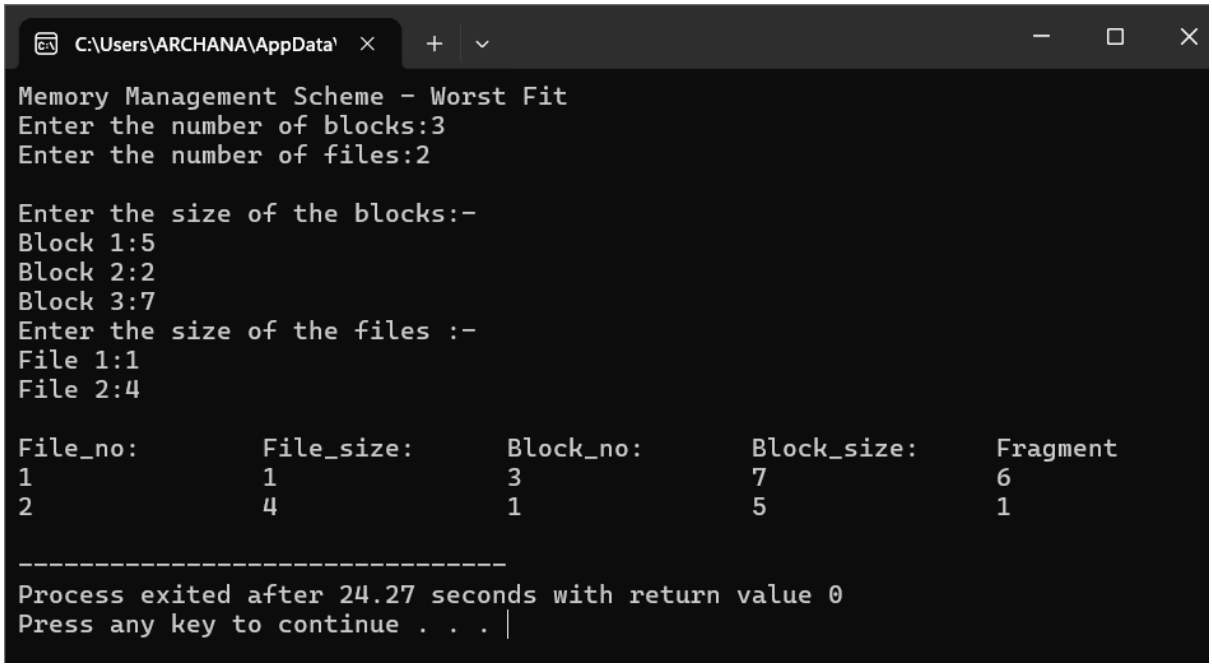
Develop a C program to implement worst fit algorithm of memory management.

PROGRAM:

```
#include <stdio.h>
int main() {
    int nb, nf, i, j;
    int block[20], file[20];
    int allocated[20] = {0};
    int blockNo[20], fragment[20];
    printf("Memory Management Scheme - Worst Fit\n");
    printf("Enter the number of blocks:");
    scanf("%d", &nb);
    printf("Enter the number of files:");
    scanf("%d", &nf);
    printf("\nEnter the size of the blocks:-\n");
    for (i = 0; i < nb; i++) {
        printf("Block %d:", i + 1);
        scanf("%d", &block[i]);
    }
    printf("Enter the size of the files :-\n");
    for (i = 0; i < nf; i++) {
        printf("File %d:", i + 1);
        scanf("%d", &file[i]);
        blockNo[i] = -1;
        fragment[i] = 0;
    }
    // Worst Fit allocation
    for (i = 0; i < nf; i++) {
        int worst = -1;
```

```
        for (j = 0; j < nb; j++) {
            if (!allocated[j] && block[j] >= file[i]) {
                if (worst == -1 ||
                    block[j] > block[worst]) {
                    worst = j;
                }
            }
        }
        if (worst != -1) {
            blockNo[i] = worst;
            fragment[i] = block[worst] - file[i];
            allocated[worst] = 1;
        }
    }
    printf("\nFile_no:\tFile_size:\tBlock_no:\tBlock_size:\tFragment\n");
    for (i = 0; i < nf; i++) {
        printf("%d\t\t%d\t\t", i + 1, file[i]);
        if (blockNo[i] != -1) {
            printf("%d\t\t%d\t\t%d\n",
                   blockNo[i] + 1,
                   block[blockNo[i]],
                   fragment[i]);
        } else {
            printf("Not Allocated\t-\t\t-\n");
        }
    }
    return 0;
}
```

OUTPUT:



QUESTION-22:

Construct a C program to implement best fit algorithm of memory management.

PROGRAM:

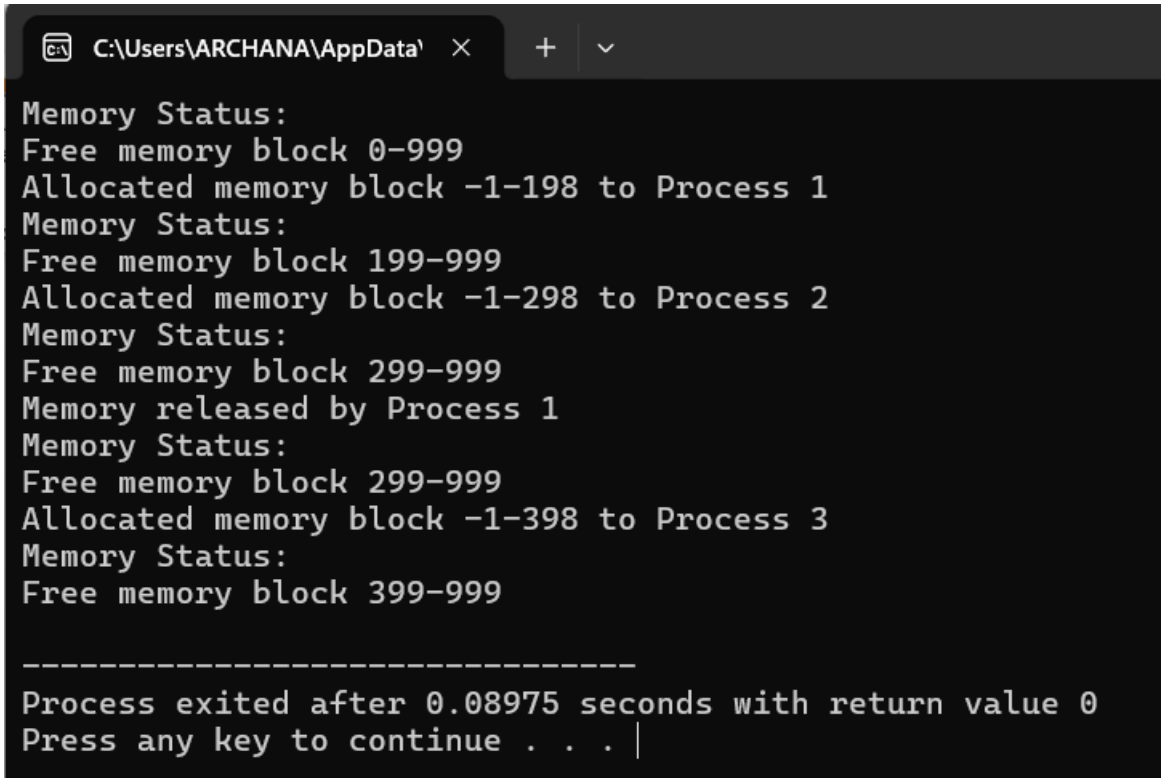
```
#include <stdio.h>
#define MAX_MEMORY 1000
int memory[MAX_MEMORY];
// Function to initialize memory
void initializeMemory() {
    for (int i = 0; i < MAX_MEMORY; i++) {
        memory[i] = -1; // -1 indicates that the memory is unallocated
    }
}
// Function to display memory status
void displayMemory() {
    int i, j;
    int count = 0;
```

```
    printf("Memory Status:\n");
    for (i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            count++;
            j = i;
            while (memory[j] == -1 && j < MAX_MEMORY) {
                j++;
            }
            printf("Free memory block %d-%d\n", i, j - 1);
            i = j - 1;
        }
    }
    if (count == 0) {
        printf("No free memory available.\n");
    }
}
// Function to allocate memory using best-fit algorithm
void allocateMemory(int processId, int size) {
    int start = -1;
    int blockSize = MAX_MEMORY;
    int bestStart = -1;
    int bestSize = MAX_MEMORY;
    for (int i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            if (blockSize == MAX_MEMORY) {
                start = i;
            }
            blockSize++;
        } else {
            if (blockSize >= size && blockSize < bestSize) {
```

```
            bestSize = blockSize;
            bestStart = start;
        }
        blockSize = 0;
    }
  }
  if (bestSize >= size) {
    for (int i = bestStart; i < bestStart + size; i++) {
      memory[i] = processId;
    }
    printf("Allocated memory block %d-%d to Process %d\n", bestStart, bestStart + size - 1, processId);
  } else {
    printf("Memory allocation for Process %d failed (not enough contiguous memory).\n", processId);
  }
}
// Function to deallocate memory
void deallocateMemory(int processId) {
  for (int i = 0; i < MAX_MEMORY; i++) {
    if (memory[i] == processId) {
      memory[i] = -1;
    }
  }
  printf("Memory released by Process %d\n", processId);
}
int main() {
  initializeMemory();
  displayMemory();
  allocateMemory(1, 200);
  displayMemory();
```

```
    allocateMemory(2, 300);
    displayMemory();
    deallocateMemory(1);
    displayMemory();
    allocateMemory(3, 400);
    displayMemory();
    return 0;
}
```

OUTPUT:



QUESTION-23:

Construct a C program to implement first fit algorithm of memory management.

PROGRAM:

```
#include <stdio.h>
#define MAX_MEMORY 1000
```

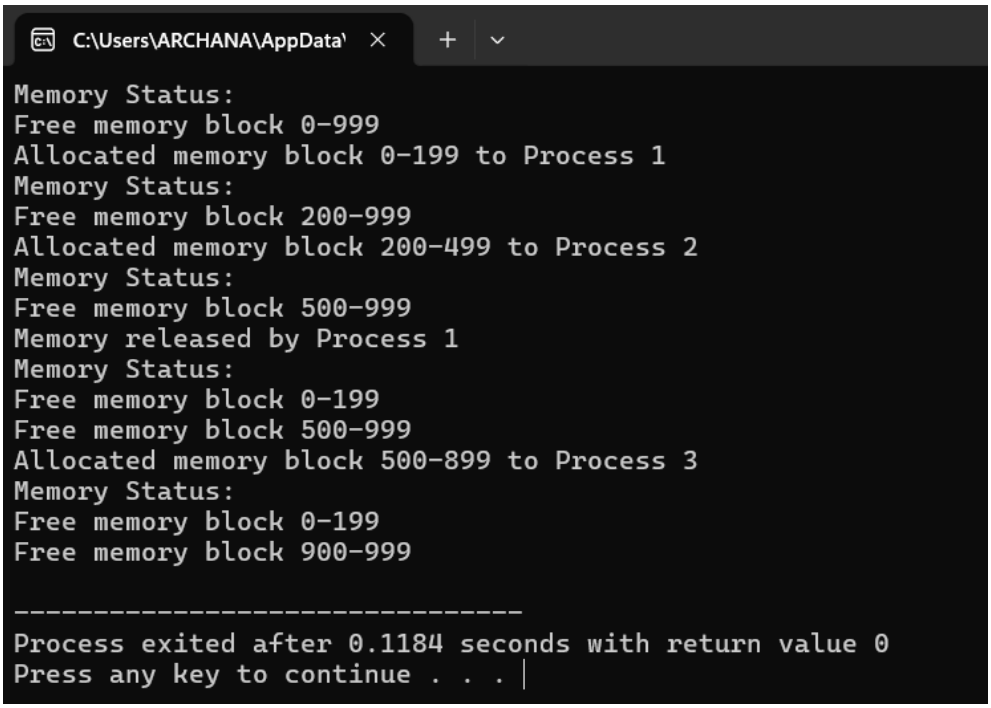
```
int memory[MAX_MEMORY];

// Function to initialize memory
void initializeMemory() {
    for (int i = 0; i < MAX_MEMORY; i++) {
        memory[i] = -1; // -1 indicates that the memory is unallocated
    }
}
// Function to display memory status
void displayMemory() {
    int i, j;
    int count = 0;
    printf("Memory Status:\n");
    for (i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            count++;
            j = i;
            while (memory[j] == -1 && j < MAX_MEMORY) {
                j++;
            }
            printf("Free memory block %d-%d\n", i, j - 1);
            i = j - 1;
        }
    }
    if (count == 0) {
        printf("No free memory available.\n");
    }
}
// Function to allocate memory using first-fit algorithm
void allocateMemory(int processId, int size) {
```

```
    int start = -1;
    int blockSize = 0;
    for (int i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == -1) {
            if (blockSize == 0) {
                start = i;
            }
            blockSize++;
        } else {
            blockSize = 0;
        }
        if (blockSize >= size) {
            break;
        }
    }
    if (blockSize >= size) {
        for (int i = start; i < start + size; i++) {
            memory[i] = processId;
        }
        printf("Allocated memory block %d-%d to Process %d\n", start, start + size - 1, processId);
    } else {
        printf("Memory allocation for Process %d failed (not enough contiguous memory).\n", processId);
    }
}
// Function to deallocate memory
void deallocateMemory(int processId) {
    for (int i = 0; i < MAX_MEMORY; i++) {
        if (memory[i] == processId) {
            memory[i] = -1;
```

```
        }
    }
    printf("Memory released by Process %d\n", processId);
}
int main() {
    initializeMemory();
    displayMemory();
    allocateMemory(1, 200);
    displayMemory();
    allocateMemory(2, 300);
    displayMemory();
    deallocateMemory(1);
    displayMemory();
    allocateMemory(3, 400);
    displayMemory();
    return 0;
}
```

OUTPUT:



QUESTION-24:

Design a C program to demonstrate UNIX system calls for file management.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
int main() { int fd;
char buffer[100];
// Creating a new file
fd = creat("sample.txt", S_IRWXU); if (fd
== -1) { perror("create"); exit(1);
} else { printf("File 'sample.txt' created successfully.\n"); close(fd);
}
// Opening an existing file for writing
fd = open("sample.txt", O_WRONLY | O_APPEND); if (fd ==
-1) { perror("open"); exit(1);
} else { printf("File 'sample.txt' opened for writing.\n");
}
// Writing data to the file
write(fd, "Hello, World!\n", 14);
printf("Data written to 'sample.txt'.\n"); close(fd);
// Opening the file for reading
fd = open("sample.txt", O_RDONLY); if (fd
== -1) { perror("open"); exit(1);
} else { printf("File 'sample.txt' opened for reading.\n"); }
// Reading data from the file
int bytesRead = read(fd, buffer, sizeof(buffer)); if
```

```
(bytesRead == -1) { perror("read"); exit(1);
} else { printf("Data read from 'sample.txt':\n"); write(STDOUT_FILENO, buffer, bytesRead);
} close(fd);
// Deleting the file
if (remove("sample.txt") == -1) { perror("remove");
exit(1);
} else { printf("File 'sample.txt' deleted.\n"); }
return 0;
}
```

OUTPUT:

